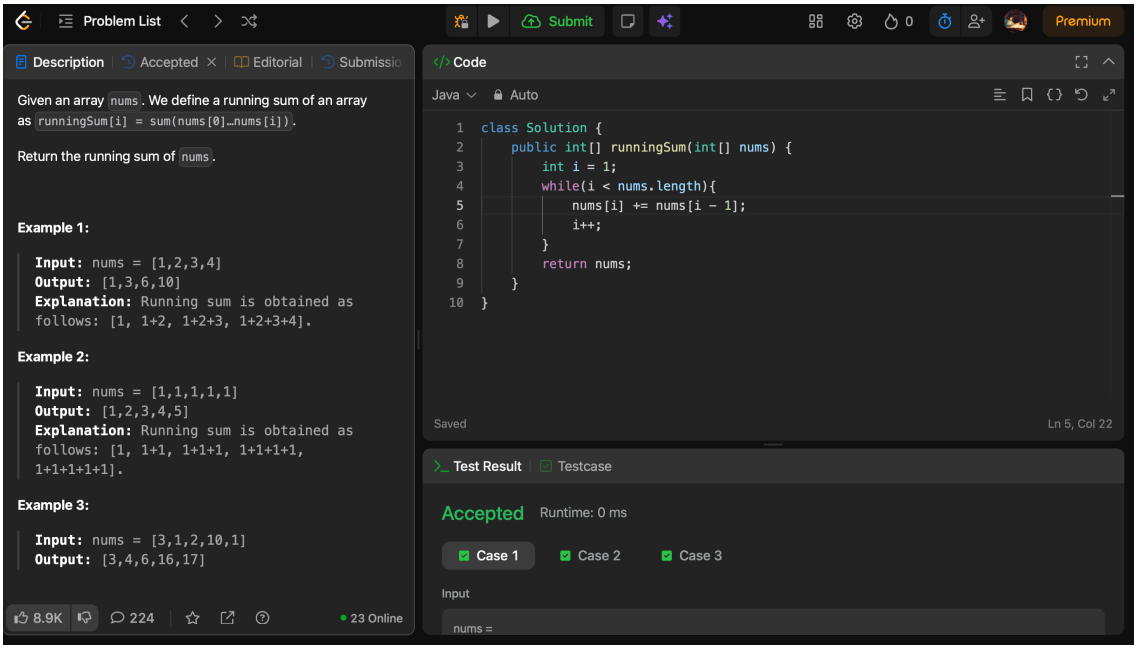
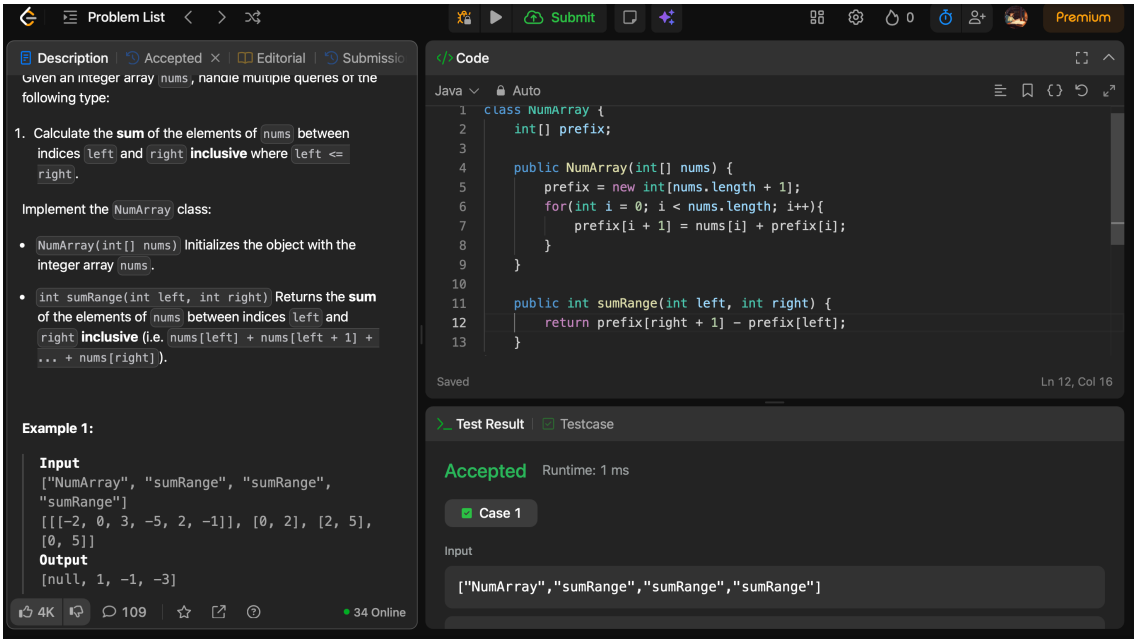
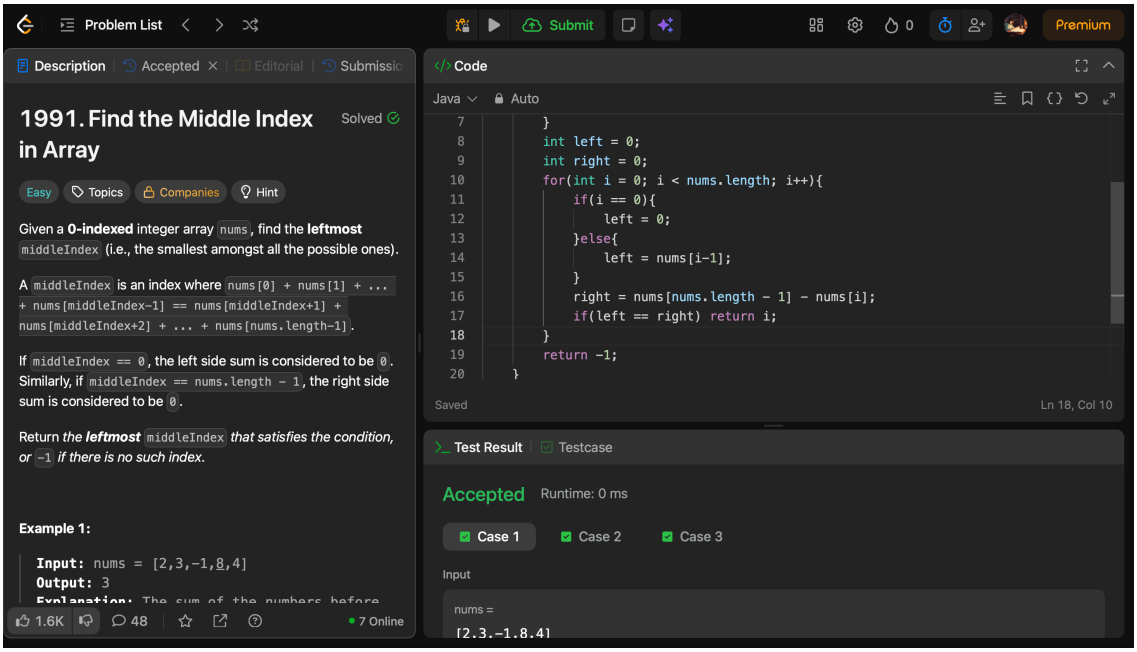
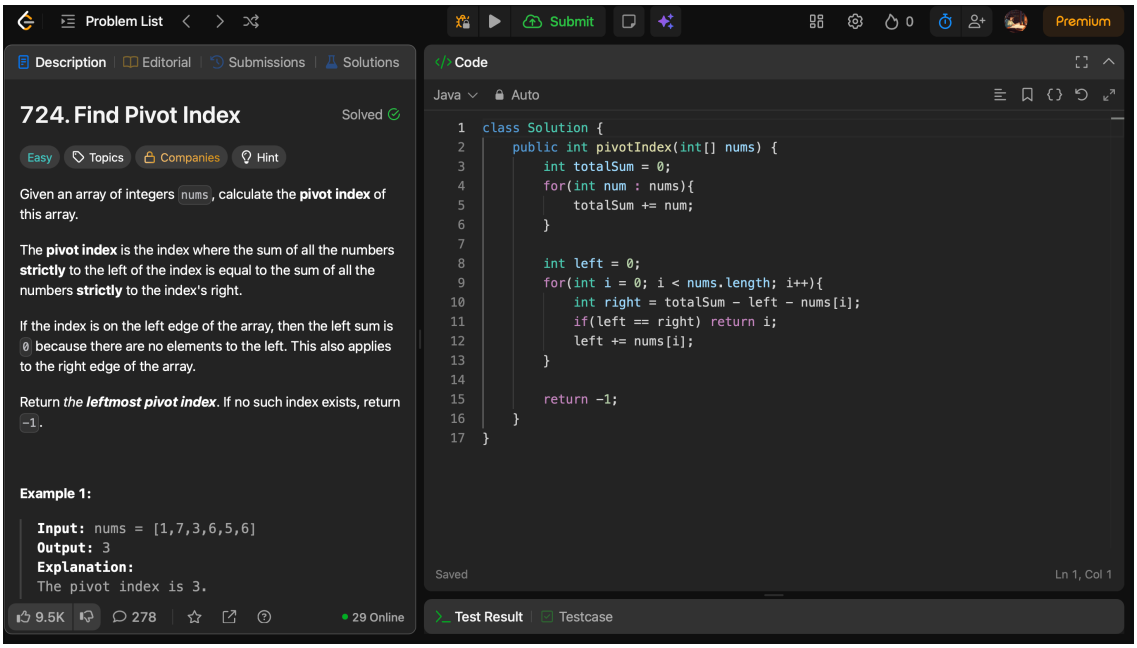
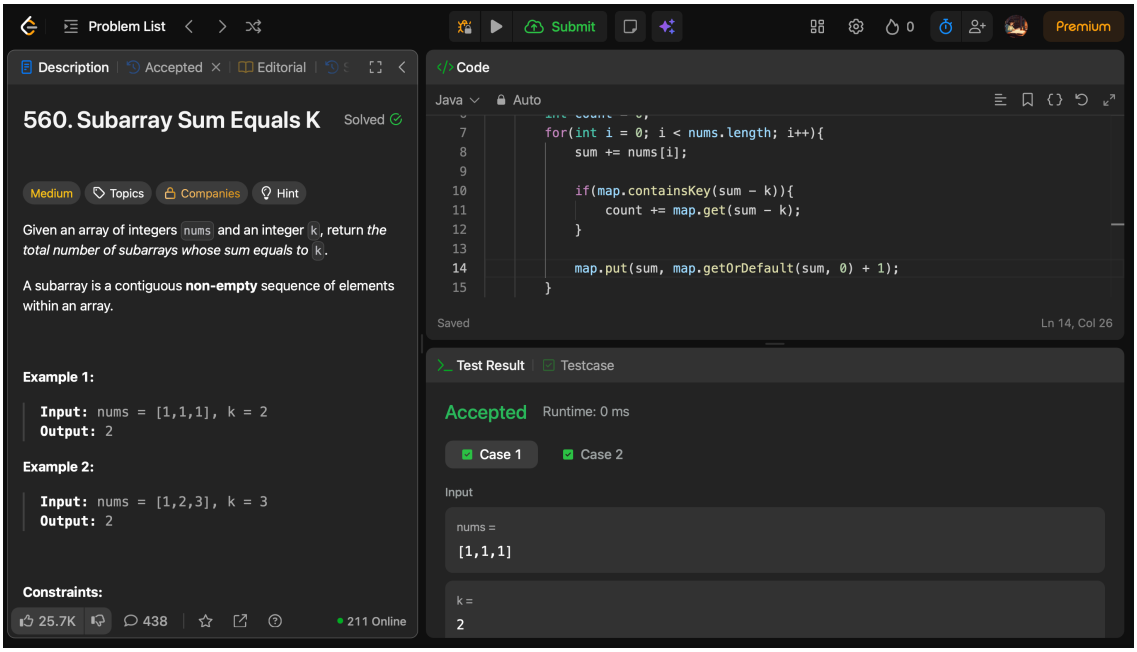
Practice Questions(Prefix Sum):

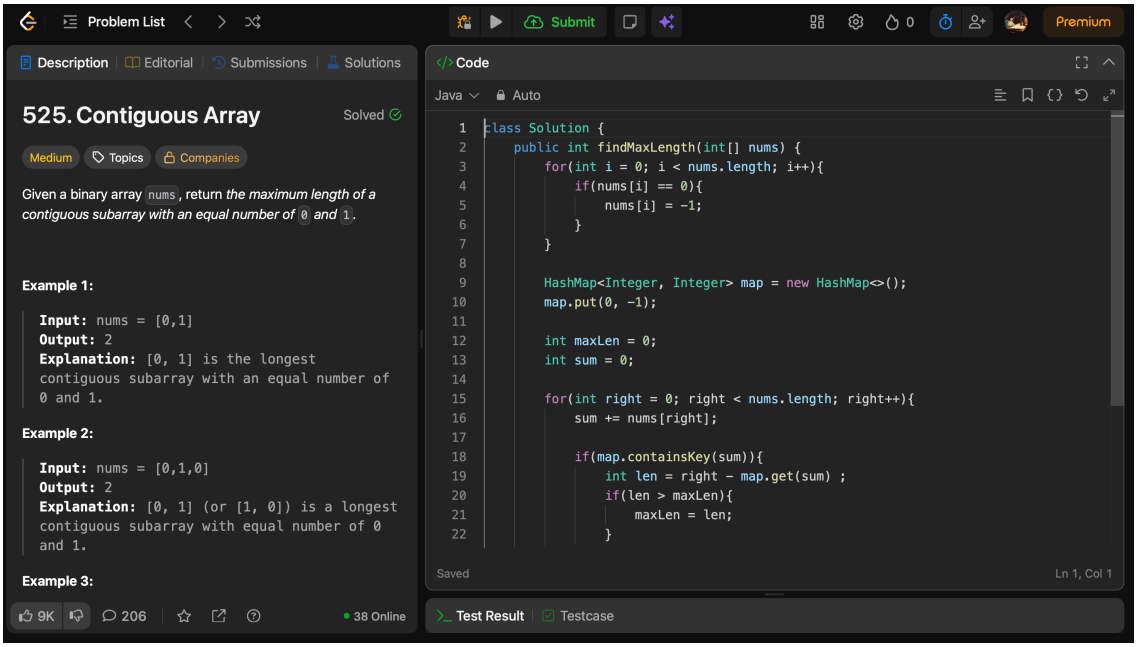


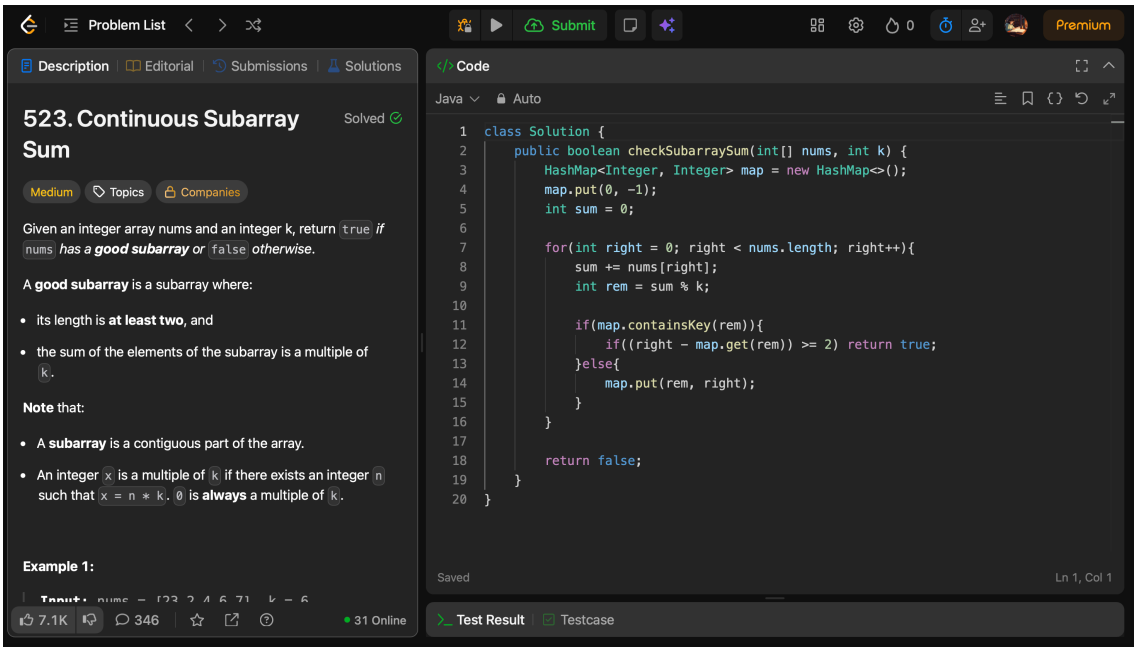


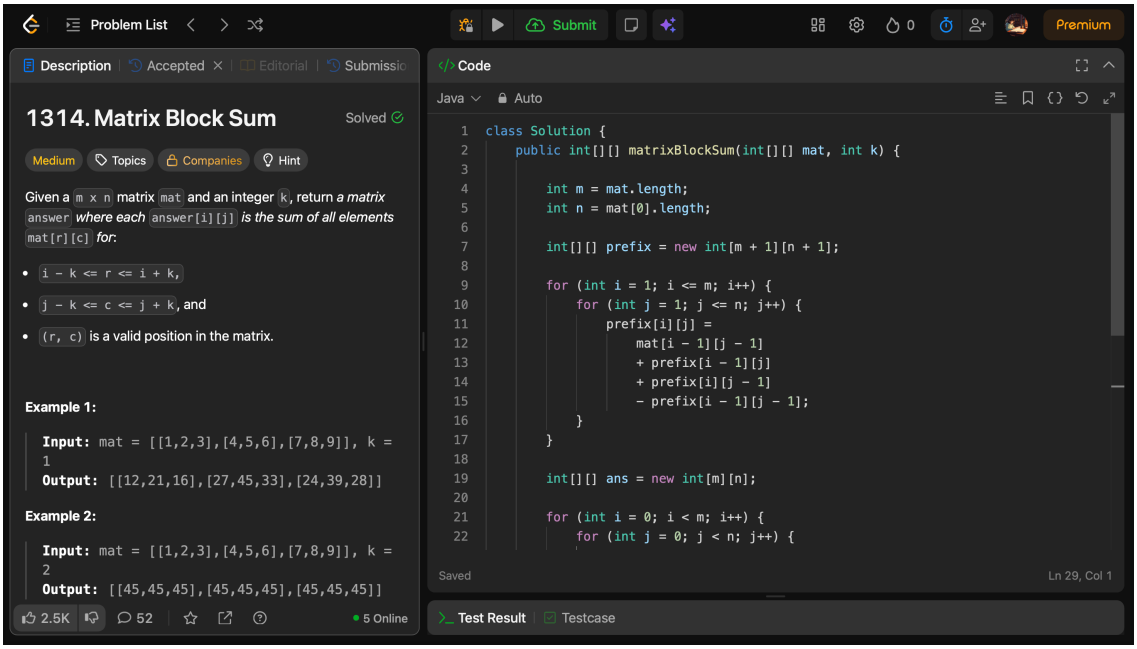

Problem List
Submit
Premium
Description
Accepted
Editorial
Submissio
1991. Find the Middle Index in Array
Solved
Easy
Topics
Companies
Hint
Given a 0-indexed integer array nums, find the leftmost middleIndex (i.e., the smallest amongst all the possible ones).
A middleIndex is an index where nums[0] + nums[1] + ... + nums[middleIndex-1] == nums[middleIndex+1] + nums[middleIndex+2] + ... + nums[nums.length-1].
If middleIndex == 0, the left side sum is considered to be 0. Similarly, if middleIndex == nums.length - 1, the right side sum is considered to be 0.
Return the leftmost middleIndex that satisfies the condition, or -1 if there is no such index.
Example 1:
Input: nums = [2,3,-1,8,4]
Output: 3
1.6K
48
7 Online
Code
Java
Auto
}
int left = 0;
int right = 0;
for(int i = 0; i < nums.length; i++){
if(i == 0){
left = 0;
}else{
left = nums[i-1];
}
right = nums[nums.length - 1] - nums[i];
if(left == right) return i;
}
return -1;
}
Saved
Ln 18, Col 10
Test Result
Testcase
Accepted
Runtime: 0 ms
Case 1
Case 2
Case 3
Input
nums =
[2,3,-1,8,4]


Problem List
Submit
Premium
Description
Editorial
Submissions
Solutions
724. Find Pivot Index
Solved
Easy
Topics
Companies
Hint
Given an array of integers nums, calculate the pivot index of this array.
The pivot index is the index where the sum of all the numbers strictly to the left of the index is equal to the sum of all the numbers strictly to the index's right.
If the index is on the left edge of the array, then the left sum is 0 because there are no elements to the left. This also applies to the right edge of the array.
Return the leftmost pivot index. If no such index exists, return -1.
Example 1:
Input: nums = [1,7,3,6,5,6]
Output: 3
Explanation:
The pivot index is 3.
9.5K
278
29 Online
Code
Java
Auto
class Solution {
public int pivotIndex(int[] nums) {
int totalSum = 0;
for(int num : nums){
totalSum += num;
}
int left = 0;
for(int i = 0; i < nums.length; i++){
int right = totalSum - left - nums[i];
if(left == right) return i;
left += nums[i];
}
return -1;
}
}
Saved
Ln 1, Col 1
Test Result
Testcase

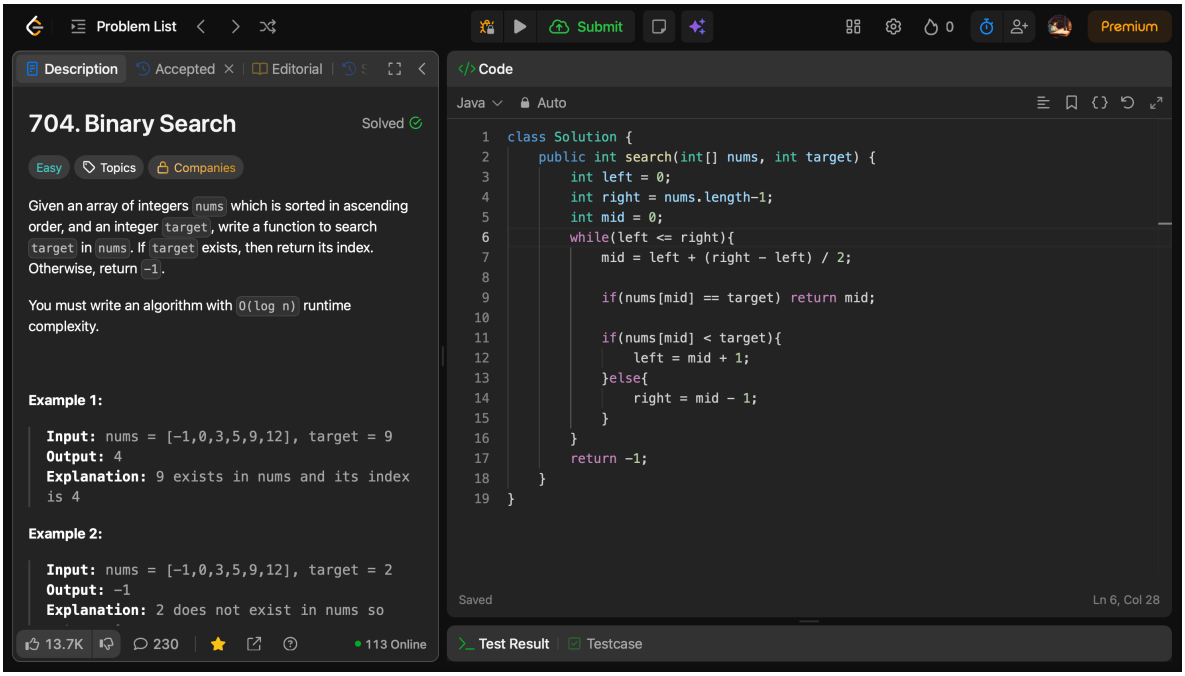

Problem List
Submit
Premium
Description
Accepted
Editorial
560. Subarray Sum Equals K
Solved
Medium
Topics
Companies
Hint
Given an array of integers nums and an integer k, return the total number of subarrays whose sum equals to k.
A subarray is a contiguous non-empty sequence of elements within an array.
Example 1:
Input: nums = [1,1,1], k = 2
Output: 2
Example 2:
Input: nums = [1,2,3], k = 3
Output: 2
Constraints:
25.7K
438
211 Online
Code
Java
Auto
for(int i = 0; i < nums.length; i++){
sum += nums[i];
if(map.containsKey(sum - k)){
count += map.get(sum - k);
}
map.put(sum, map.getOrDefault(sum, 0) + 1);
}
Saved
Ln 14, Col 26
Test Result
Testcase
Accepted
Runtime: 0 ms
Case 1
Case 2
Input
nums =
[1,1,1]
k =
2

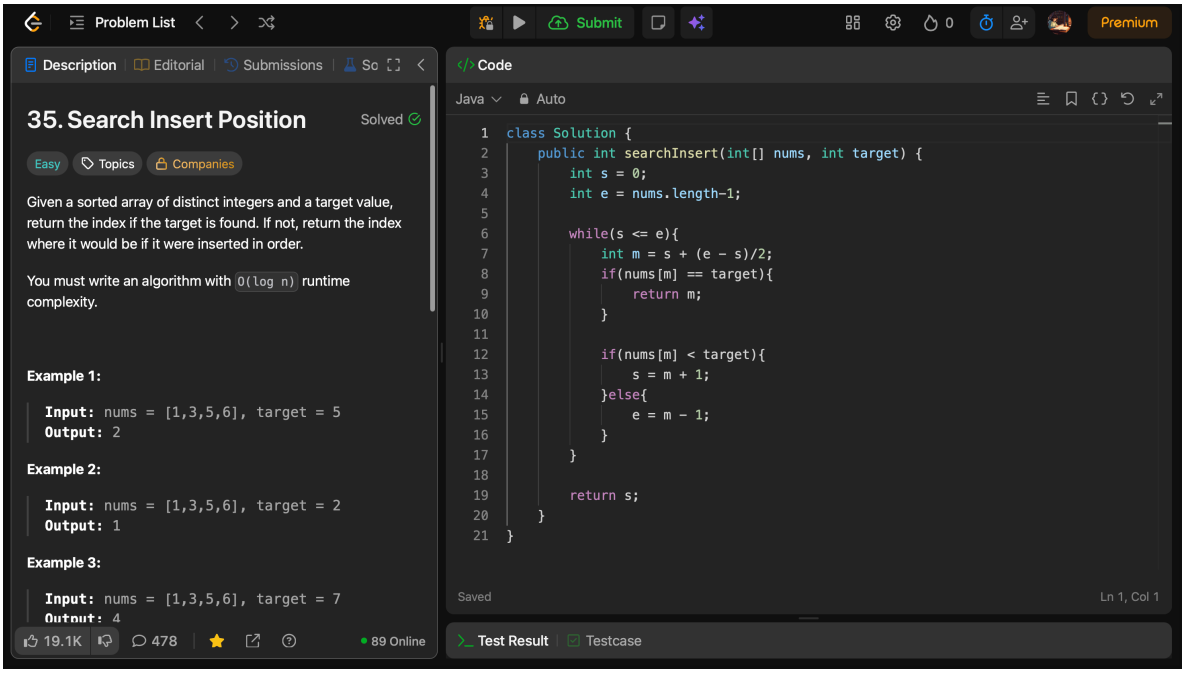

Problem List
Submit
Premium
Description
Editorial
Submissions
Solutions
525. Contiguous Array
Solved
Medium
Topics
Companies
Given a binary array nums, return the maximum length of a contiguous subarray with an equal number of 0 and 1.
Example 1:
Input: nums = [0,1]
Output: 2
Explanation: [0, 1] is the longest contiguous subarray with an equal number of 0 and 1.
Example 2:
Input: nums = [0,1,0]
Output: 2
Explanation: [0, 1] (or [1, 0]) is a longest contiguous subarray with equal number of 0 and 1.
Example 3:
9K
206
38 Online
Code
Java
Auto
class Solution {
public int findMaxLength(int[] nums) {
for(int i = 0; i < nums.length; i++){
if(nums[i] == 0){
nums[i] = -1;
}
}
HashMap<Integer, Integer> map = new HashMap<>();
map.put(0, -1);
int maxLen = 0;
int sum = 0;
for(int right = 0; right < nums.length; right++){
sum += nums[right];
if(map.containsKey(sum)){
int len = right - map.get(sum) ;
if(len > maxLen){
maxLen = len;
}
Saved
Ln 1, Col 1
Test Result
Testcase

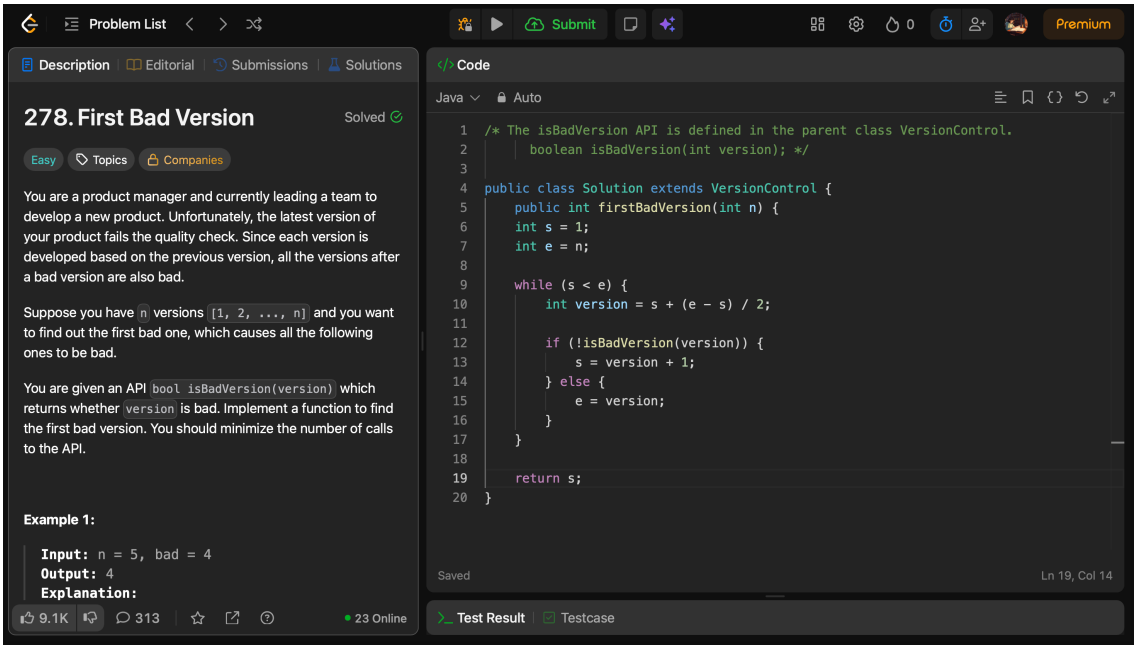


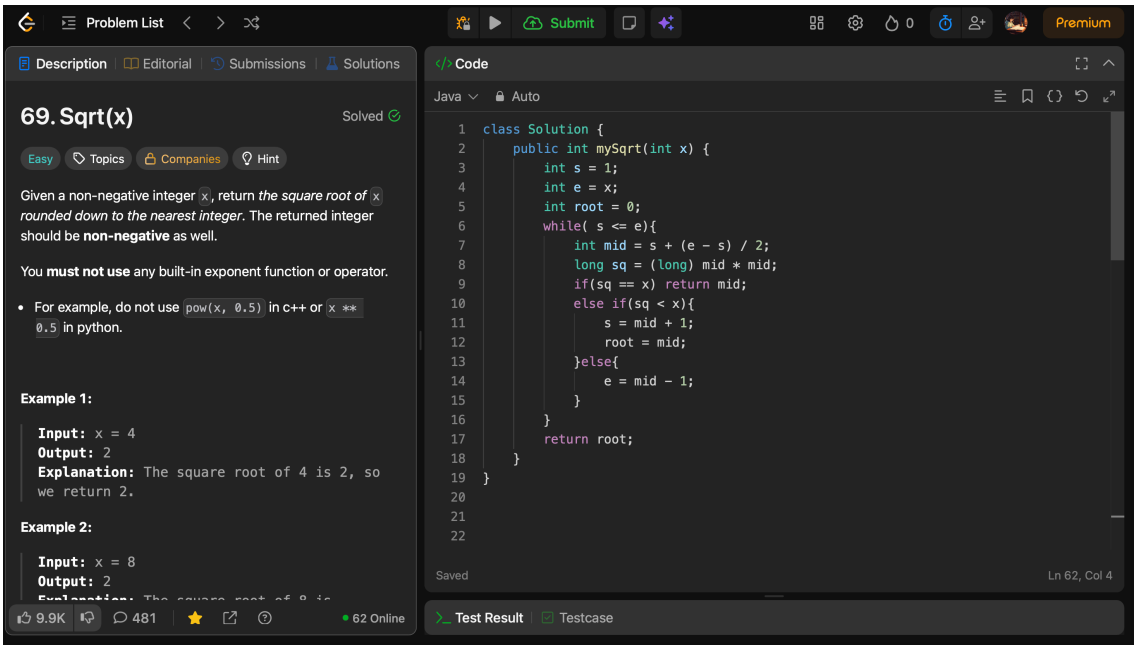


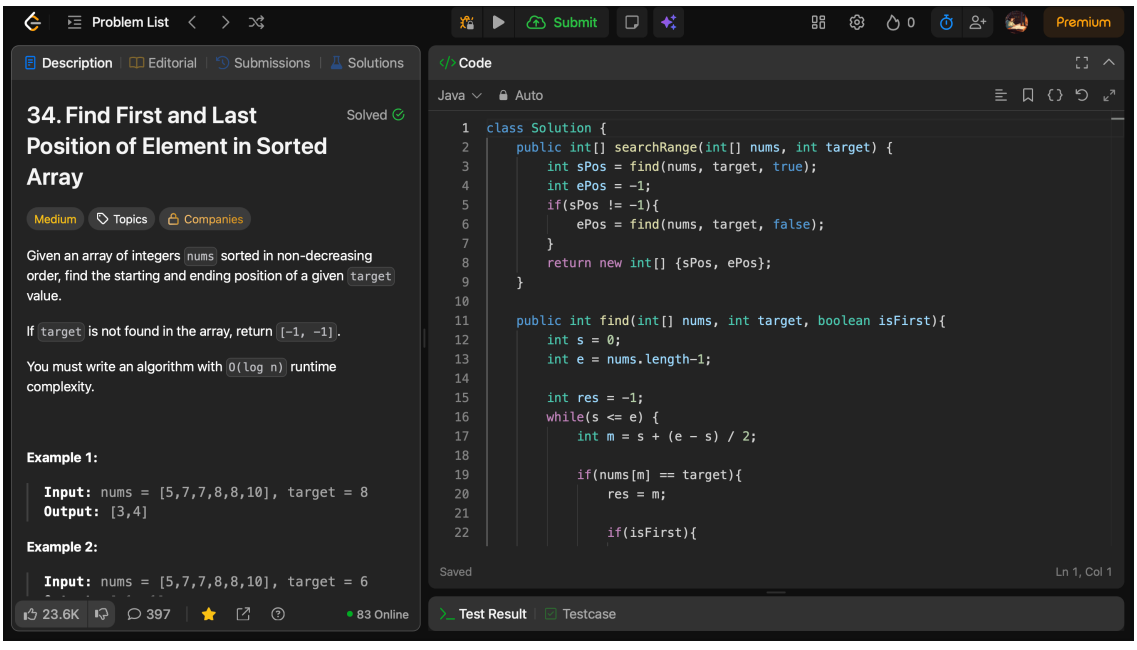
Practice Questions(BS):


Problem List
Submit
Premium
Description
Accepted
Editorial
Code
Java
Auto
704. Binary Search
Solved
Easy
Topics
Companies
Given an array of integers nums which is sorted in ascending order, and an integer target, write a function to search target in nums. If target exists, then return its index. Otherwise, return -1.
You must write an algorithm with O(log n) runtime complexity.
Example 1:
Input: nums = [-1,0,3,5,9,12], target = 9
Output: 4
Explanation: 9 exists in nums and its index is 4
Example 2:
Input: nums = [-1,0,3,5,9,12], target = 2
Output: -1
Explanation: 2 does not exist in nums so
13.7K
230
113 Online
class Solution {
public int search(int[] nums, int target) {
int left = 0;
int right = nums.length-1;
int mid = 0;
while(left <= right){
mid = left + (right - left) / 2;
if(nums[mid] == target) return mid;
if(nums[mid] < target){
left = mid + 1;
}else{
right = mid - 1;
}
}
return -1;
}
}
Saved
Ln 6, Col 28
Test Result
Testcase


Problem List
Submit
Premium
Description
Editorial
Submissions
Code
Java
Auto
35. Search Insert Position
Solved
Easy
Topics
Companies
Given a sorted array of distinct integers and a target value, return the index if the target is found. If not, return the index where it would be if it were inserted in order.
You must write an algorithm with O(log n) runtime complexity.
Example 1:
Input: nums = [1,3,5,6], target = 5
Output: 2
Example 2:
Input: nums = [1,3,5,6], target = 2
Output: 1
Example 3:
Input: nums = [1,3,5,6], target = 7
19.1K
478
89 Online
class Solution {
public int searchInsert(int[] nums, int target) {
int s = 0;
int e = nums.length-1;
while(s <= e){
int m = s + (e - s)/2;
if(nums[m] == target){
return m;
}
if(nums[m] < target){
s = m + 1;
}else{
e = m - 1;
}
}
return s;
}
}
Saved
Ln 1, Col 1
Test Result
Testcase


Problem List
Submit
Premium
Description
Editorial
Submissions
Solutions
278. First Bad Version
Solved
Easy
Topics
Companies
You are a product manager and currently leading a team to develop a new product. Unfortunately, the latest version of your product fails the quality check. Since each version is developed based on the previous version, all the versions after a bad version are also bad.
Suppose you have n versions [1, 2, ..., n] and you want to find out the first bad one, which causes all the following ones to be bad.
You are given an API bool isBadVersion(version) which returns whether version is bad. Implement a function to find the first bad version. You should minimize the number of calls to the API.
Example 1:
Input: n = 5, bad = 4
Output: 4
Explanation:
9.1K
313
23 Online
Code
Java
Auto
/* The isBadVersion API is defined in the parent class VersionControl.
boolean isBadVersion(int version); */
public class Solution extends VersionControl {
public int firstBadVersion(int n) {
int s = 1;
int e = n;
while (s < e) {
int version = s + (e - s) / 2;
if (!isBadVersion(version)) {
s = version + 1;
} else {
e = version;
}
}
return s;
}
Saved
Ln 19, Col 14
Test Result
Testcase


Problem List
Submit
Premium
Description
Editorial
Submissions
Solutions
69. Sqrt(x)
Solved
Easy
Topics
Companies
Hint
Given a non-negative integer x, return the square root of x rounded down to the nearest integer. The returned integer should be non-negative as well.
You must not use any built-in exponent function or operator.
For example, do not use pow(x, 0.5) in c++ or x ** 0.5 in python.
Example 1:
Input: x = 4
Output: 2
Explanation: The square root of 4 is 2, so we return 2.
Example 2:
Input: x = 8
Output: 2
9.9K
481
62 Online
Code
Java
Auto
class Solution {
public int mySqrt(int x) {
int s = 1;
int e = x;
int root = 0;
while(s <= e){
int mid = s + (e - s) / 2;
long sq = (long) mid * mid;
if(sq == x) return mid;
else if(sq < x){
s = mid + 1;
root = mid;
}else{
e = mid - 1;
}
}
return root;
}
}
Saved
Ln 62, Col 4
Test Result
Testcase


Problem List
Submit
Premium
Description
Editorial
Submissions
Solutions
34. Find First and Last Position of Element in Sorted Array
Solved
Medium
Topics
Companies
Given an array of integers nums sorted in non-decreasing order, find the starting and ending position of a given target value.
If target is not found in the array, return [-1, -1].
You must write an algorithm with O(log n) runtime complexity.
Example 1:
Input: nums = [5,7,7,8,8,10], target = 8
Output: [3,4]
Example 2:
Input: nums = [5,7,7,8,8,10], target = 6
23.6K
397
83 Online
Code
Java
Auto
class Solution {
public int[] searchRange(int[] nums, int target) {
int sPos = find(nums, target, true);
int ePos = -1;
if(sPos != -1){
ePos = find(nums, target, false);
}
return new int[] {sPos, ePos};
}
public int find(int[] nums, int target, boolean isFirst){
int s = 0;
int e = nums.length-1;
int res = -1;
while(s <= e) {
int m = s + (e - s) / 2;
if(nums[m] == target){
res = m;
if(isFirst){
Saved
Ln 1, Col 1
Test Result
Testcase

Problem List

Description | Editorial | Submissions | Solutions

33. Search in Rotated Sorted Array

Solved

Medium | Topics | Companies

There is an integer array `nums` sorted in ascending order (with **distinct** values).

Prior to being passed to your function, `nums` is **possibly left rotated** at an unknown index `k` (`1 <= k < nums.length`) such that the resulting array is `[nums[k], nums[k+1], ..., nums[n-1], nums[0], nums[1], ..., nums[k-1]]` (**0-indexed**). For example, `[0,1,2,4,5,6,7]` might be left rotated by `3` indices and become `[4,5,6,7,0,1,2]`.

Given the array `nums` **after** the possible rotation and an integer `target`, return *the index of* `target` *if it is in* `nums`, *or* `-1` *if it is not in* `nums`.

You must write an algorithm with `O(log n)` runtime complexity.

Example 1:

30.7K | 717 | 234 Online

Submit | 0 | Premium

Code

Java | Auto

```java
class Solution {
    public int search(int[] nums, int target) {
        int s = 0;
        int e = nums.length-1;

        while(s <= e){
            int m = s + (e - s) / 2;

            if(nums[m] == target) return m;

            if(nums[m] >= nums[s]){
                if(target >= nums[s] && nums[m] > target){
                    e = m - 1;
                }else{
                    s = m + 1;
                }
            }else{
                if(target > nums[m] && nums[e] >= target){
                    s = m + 1;
                }else{
                    e = m - 1;
                }
```

Saved | Ln 27, Col 2

Test Result | Testcase

Problem List

Description | Editorial | Submissions | Solutions

162. Find Peak Element

Solved

Medium | Topics | Companies

A peak element is an element that is strictly greater than its neighbors.

Given a **0-indexed** integer array `nums`, find a peak element, and return its index. If the array contains multiple peaks, return the index to **any of the peaks**.

You may imagine that `nums[-1] = nums[n] = -∞`. In other words, an element is always considered to be strictly greater than a neighbor that is outside the array.

You must write an algorithm that runs in `O(log n)` time.

Example 1:

```
Input: nums = [1,2,3,1]
Output: 2
Explanation: 3 is a peak element and your function should return the index number 2.
```

14.9K | 566 | 64 Online

Submit | 0 | Premium

Code

Java | Auto

```java
class Solution {
    public int findPeakElement(int[] nums) {
        int s = 0;
        int e = nums.length-1;

        while(s <= e) {
            int m = s + (e - s) / 2;

            if(m+1 < nums.length && nums[m] < nums[m+1]){
                s = m + 1;
            }else{
                e = m - 1;
            }
        }

        return s;
    }
}
```

Restored from local | Upgrade to Cloud Saving | Ln 1, Col 1

Test Result | Testcase

Problem List

Description | Editorial | Submissions | So

74. Search a 2D Matrix

Solved

Medium | Topics | Companies

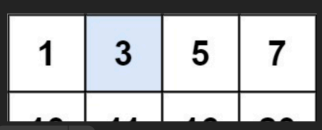
You are given an `m x n` integer matrix `matrix` with the following two properties:

- Each row is sorted in non-decreasing order.
- The first integer of each row is greater than the last integer of the previous row.

Given an integer `target`, return `true` *if* `target` *is in* `matrix` *or* `false` *otherwise*.

You must write a solution in `O(log(m * n))` time complexity.

Example 1:



18.2K | 405 | 76 Online

Submit | 0 | Premium

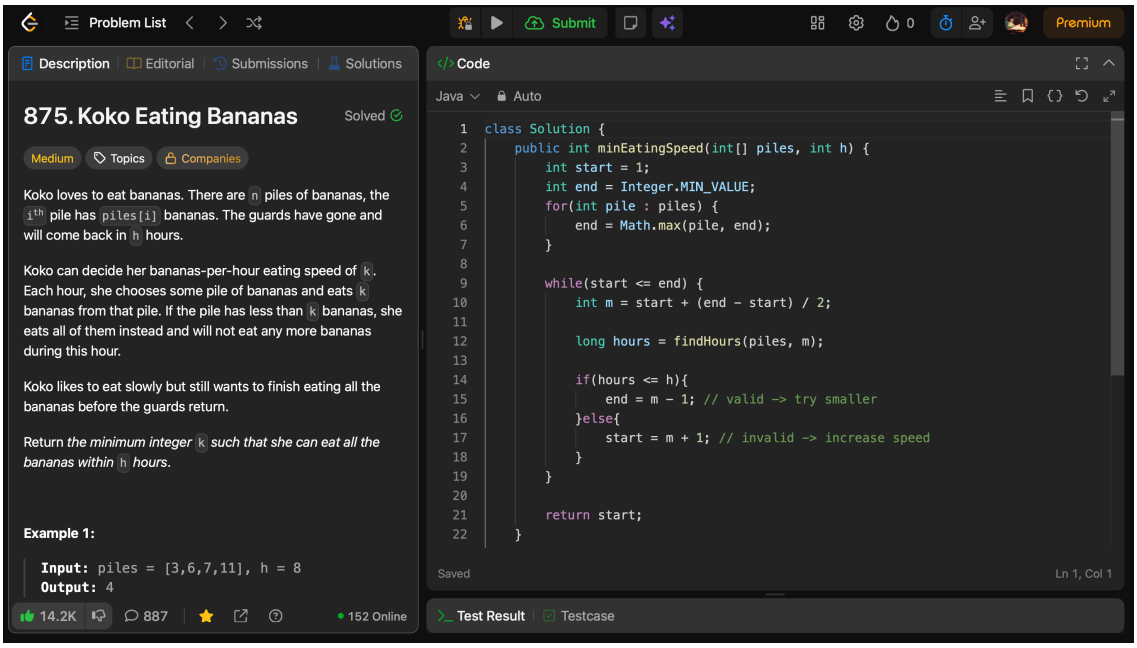
Code

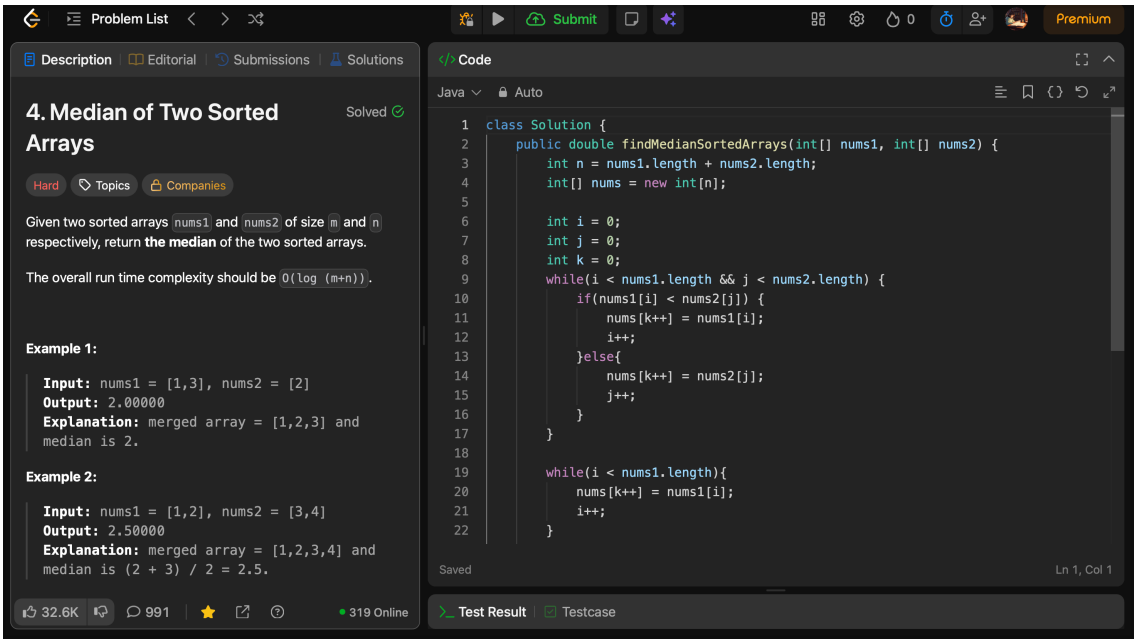
Java | Auto

```java
            return search(matrix, target, rStart + 1, cMid + 1, cols-1);
        }
        return false;
    }


    public boolean search(int[][] mat , int target, int row, int cStart, int cEnd)
    {
        while(cStart <= cEnd) {
            int mid = cStart + (cEnd - cStart) / 2;
            if(mat[row][mid] == target){
                return true;
            }
            if(mat[row][mid] < target) {
                cStart = mid + 1;
            }else{
                cEnd = mid - 1;
            }
        }

        return false;
    }
}
```

Saved | Ln 63, Col 2

Test Result | Testcase


Problem List
Submit
Premium
Description
Editorial
Submissions
Solutions
875. Koko Eating Bananas
Solved
Medium
Topics
Companies
Koko loves to eat bananas. There are n piles of bananas, the i^th pile has piles[i] bananas. The guards have gone and will come back in h hours.
Koko can decide her bananas-per-hour eating speed of k. Each hour, she chooses some pile of bananas and eats k bananas from that pile. If the pile has less than k bananas, she eats all of them instead and will not eat any more bananas during this hour.
Koko likes to eat slowly but still wants to finish eating all the bananas before the guards return.
Return the minimum integer k such that she can eat all the bananas within h hours.
Example 1:
Input: piles = [3,6,7,11], h = 8
Output: 4
14.2K
887
152 Online
Code
Java
Auto
class Solution {
public int minEatingSpeed(int[] piles, int h) {
int start = 1;
int end = Integer.MIN_VALUE;
for(int pile : piles) {
end = Math.max(pile, end);
}
while(start <= end) {
int m = start + (end - start) / 2;
long hours = findHours(piles, m);
if(hours <= h){
end = m - 1; // valid -> try smaller
}else{
start = m + 1; // invalid -> increase speed
}
}
return start;
}
Saved
Ln 1, Col 1
Test Result
Testcase


Problem List
Submit
Premium
Description
Editorial
Submissions
Solutions
4. Median of Two Sorted Arrays
Solved
Hard
Topics
Companies
Given two sorted arrays nums1 and nums2 of size m and n respectively, return the median of the two sorted arrays.
The overall run time complexity should be O(log (m+n)).
Example 1:
Input: nums1 = [1,3], nums2 = [2]
Output: 2.00000
Explanation: merged array = [1,2,3] and median is 2.
Example 2:
Input: nums1 = [1,2], nums2 = [3,4]
Output: 2.50000
Explanation: merged array = [1,2,3,4] and median is (2 + 3) / 2 = 2.5.
32.6K
991
319 Online
Code
Java
Auto
class Solution {
public double findMedianSortedArrays(int[] nums1, int[] nums2) {
int n = nums1.length + nums2.length;
int[] nums = new int[n];
int i = 0;
int j = 0;
int k = 0;
while(i < nums1.length && j < nums2.length) {
if(nums1[i] < nums2[j]) {
nums[k++] = nums1[i];
i++;
}else{
nums[k++] = nums2[j];
j++;
}
}
while(i < nums1.length){
nums[k++] = nums1[i];
i++;
}
Saved
Ln 1, Col 1
Test Result
Testcase